Árboles generales y binarios

En estructuras de datos, un árbol es una forma de organizar información en nodos que se conectan entre sí. A diferencia de las listas enlazadas, aquí cada nodo puede tener varios hijos.

Árboles generales

Un árbol general (o n-ario) es aquel en el que cada nodo puede tener cualquier cantidad de hijos. No hay un límite fijo. Se usan cuando la estructura de los datos no es regular, por ejemplo en sistemas de archivos o en la representación de jerarquías organizacionales.

Árboles binarios

Un árbol binario es más restringido: cada nodo puede tener como máximo dos hijos, el izquierdo y el derecho. Esta limitación hace que sean más fáciles de manejar y que existan algoritmos eficientes para recorrerlos y buscar elementos.

Los recorridos más comunes en un árbol binario son tres:

Preorden: primero se visita el nodo actual, después el subárbol izquierdo y por último el derecho.

Inorden: primero el subárbol izquierdo, luego el nodo actual y al final el derecho. En un árbol binario de búsqueda este recorrido da los elementos ordenados.

Posorden: primero se recorre el subárbol izquierdo, después el derecho y al final el nodo actual. Se usa mucho cuando hay que liberar memoria de los nodos.

Código en C++

Aquí va un ejemplo simple de un árbol binario con los tres recorridos. Usé una estructura de nodo con punteros y funciones recursivas, que es la forma más directa de hacerlo.

```
#include <iostream>
using namespace std;

struct Nodo {
    int dato;
    Nodo* izq;
    Nodo* der;

    Nodo(int valor) {
        dato = valor;
        izq = nullptr;
        der = nullptr;
    }
};
```

```
void preorden(Nodo* nodo) {
    if (nodo == nullptr) return;
    cout << nodo->dato << " ";
    preorden(nodo->izq);
    preorden(nodo->der);
}

void inorden(Nodo* nodo) {
    if (nodo == nullptr) return;
    inorden(nodo->izq);
    cout << nodo->dato << " ";
    inorden(nodo->der);
}

void posorden(Nodo* nodo) {
    if (nodo == nullptr) return;
    posorden(nodo->izq);
    posorden(nodo->der);
    cout << nodo->dato << " ";
}

int main() {
    // Creo un arbol de ejemplo
    //       1
    //      / \\
    //     2   3
    //    / \\
    //   4   5

    Nodo* raiz = new Nodo(1);
    raiz->izq = new Nodo(2);
    raiz->der = new Nodo(3);
    raiz->izq->izq = new Nodo(4);
    raiz->izq->der = new Nodo(5);

    cout << "Preorden:  ";
    preorden(raiz);
    cout << endl;

    cout << "Inorden:   ";
    inorden(raiz);
    cout << endl;
```

```
    cout << "Posorden:  ";
    posorden(raiz);
    cout << endl;

    return 0;
}
```

Si corres el programa, la salida es:

```
Preorden:  1 2 4 5 3
Inorden:   4 2 5 1 3
Posorden:  4 5 2 3 1
```

El código es bastante directo. Cada función chequea si el nodo es nulo y si no, aplica el orden correspondiente. Para liberar la memoria se podría hacer un recorrido posorden y borrar los nodos, pero para este ejemplo no era necesario.